MongoDB Students CRUD System

NoSQL Databases Activity — System Documentation
Submitted by: _____ Date: _____

1. System Overview

This system is a **Student Records CRUD application** built with **Node.js** and **MongoDB** (a document-oriented NoSQL database). It demonstrates the four basic database operations — **Create, Read, Update, and Delete** — in two forms:

• **Console demo** (crud-demo.js) — a script that walks through each CRUD operation on a schoolDB.students collection and prints the result of every step.

• **Web application** — a browser UI (public/index.html) that talks to a REST API (api/students.js) so users can add, view, edit, and delete student records from a web page. The same API code is written as a Vercel-style serverless function, so it can be deployed to the cloud with MongoDB Atlas.

2. Technology Stack and Project Structure

Backend: Node.js + official mongodb driver | **Database:** MongoDB 8 (Docker container, port 27017) | **Frontend:** plain HTML/CSS/JavaScript

File	Purpose
`crud-demo.js`	Console walkthrough of all four CRUD operations
`api/students.js`	REST API: GET (read, with filters), POST (create), PUT (update), DELETE
`lib/mongodb.js`	Shared, cached MongoDB connection (env-var based, Atlas-ready)
`public/index.html`	Web UI: add-form, students table, edit/delete buttons
`local-dev.js`	Local server on port 3000 (serves UI + API, mimics Vercel)
`test-connection.js`	Diagnostic tool that validates a MongoDB connection string

3. The Database

MongoDB stores data as **documents** (JSON-like objects) inside **collections** instead of rows in tables. This system uses one database, **schoolDB**, with one collection, **students**. There is no fixed schema — each document gets an auto-generated **_id** (ObjectId) and can hold nested data such as the subjects array. Example document:

```
{
  "_id": ObjectId("6aed77be3330be6b59f2b7be"),
  "name": "Juan Dela Cruz",
  "age": 20,
  "course": "BSCS",
  "subjects": ["Programming", "Databases"],
  "enrolled": true,
  "yearLevel": 2
}
```

The four CRUD operations map to these driver methods: **insertOne/insertMany** (Create), **find/findOne** with filters, operators like $gt and projections (Read), **updateOne/updateMany** with $set and $push (Update), and **deleteOne/deleteMany** (Delete).

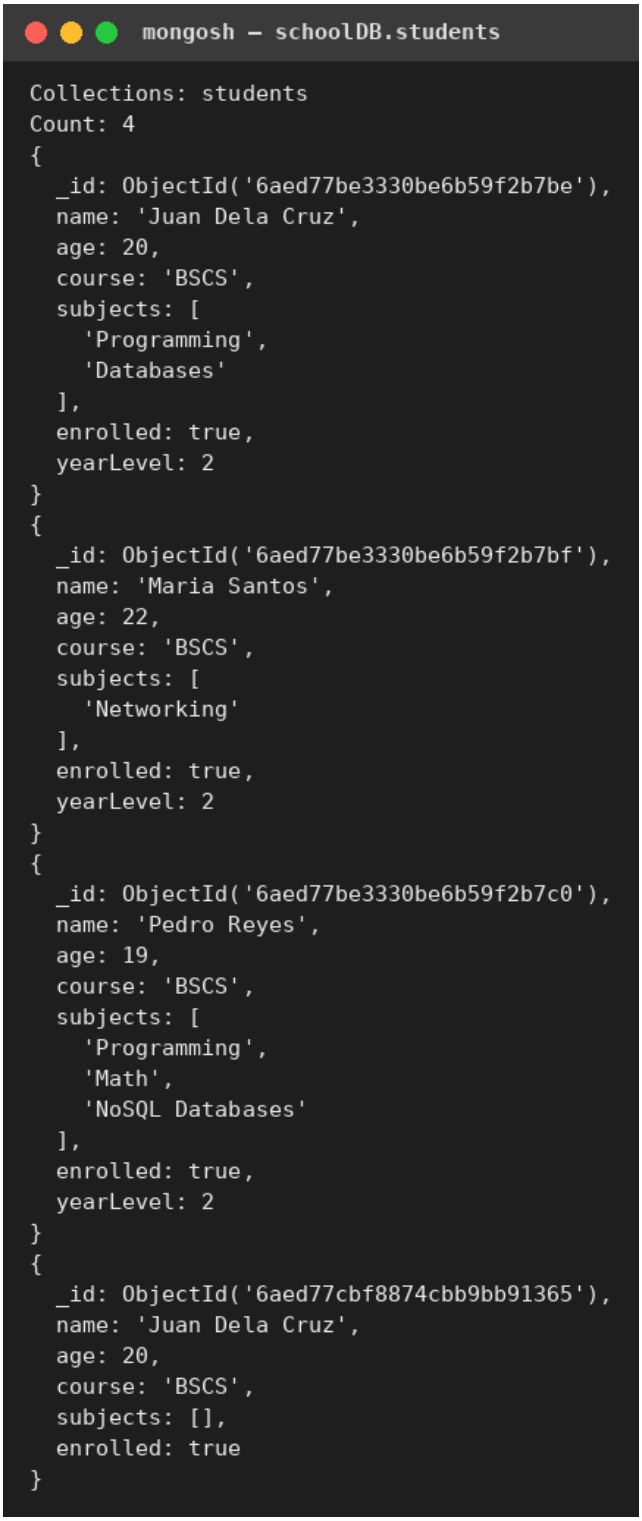
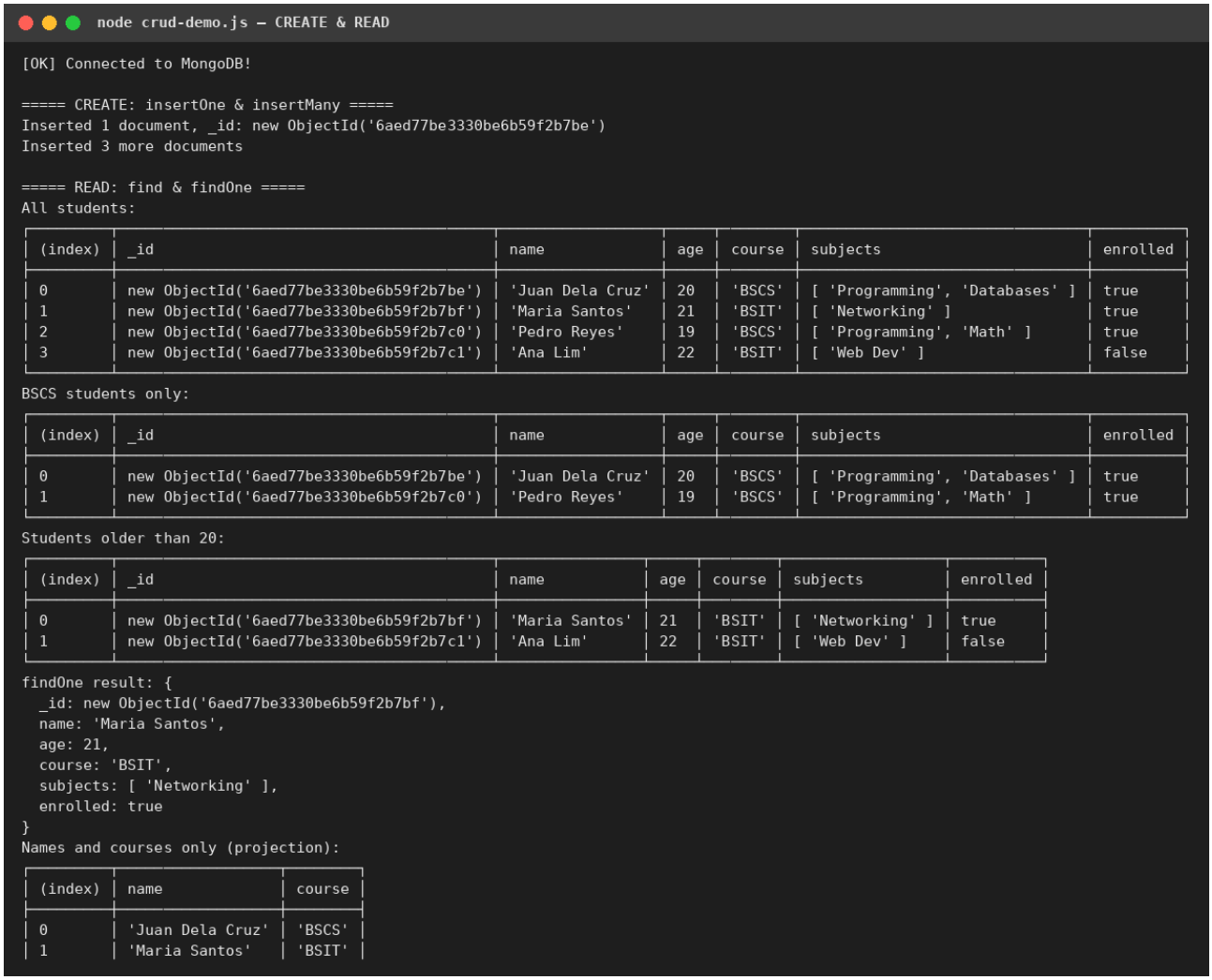


Figure 1 — Actual data inside MongoDB, queried through the mongosh shell: the schoolDB database, its students collection, and the stored documents.

4. Running the System — Console CRUD Demo

Running `node crud-demo.js` connects to MongoDB and performs every CRUD operation step by step:



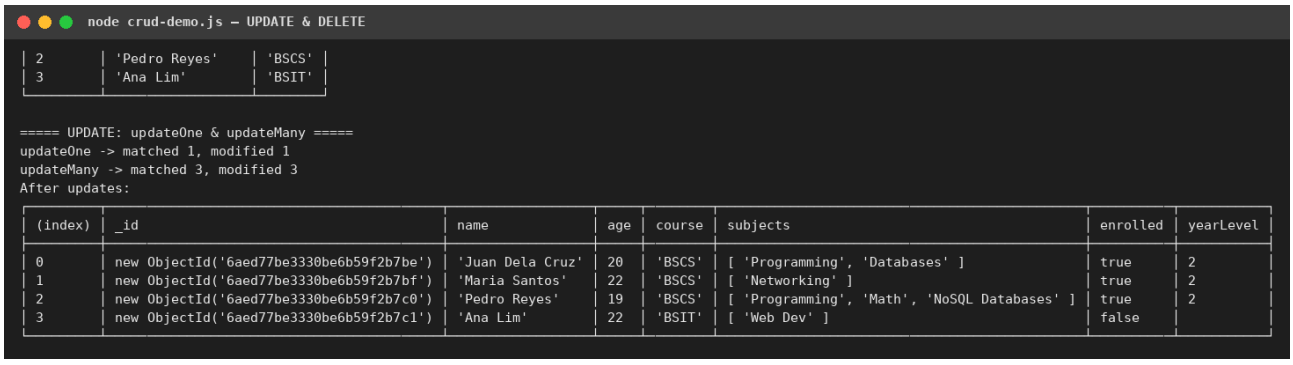
```
node crud-demo.js — CREATE & READ

[OK] Connected to MongoDB!

===== CREATE: insertOne & insertMany =====
Inserted 1 document, _id: new ObjectId('6aed77be3330be6b59f2b7be')
Inserted 3 more documents

===== READ: find & findOne =====
All students:
┌─────────┬──────────────────────────────────────────┬──────────────┬─────┬──────────┬──────────────────────────────┬──────────┐
│ (index) │ _id                                      │ name         │ age │ course   │ subjects                     │ enrolled │
├─────────┼──────────────────────────────────────────┼──────────────┼─────┼──────────┼──────────────────────────────┼──────────┤
│ 0       │ new ObjectId('6aed77be3330be6b59f2b7be') │ 'Juan Dela Cruz' │ 20  │ 'BSCS'   │ [ 'Programming', 'Databases' ] │ true     │
│ 1       │ new ObjectId('6aed77be3330be6b59f2b7bf') │ 'Maria Santos'   │ 21  │ 'BSIT'   │ [ 'Networking' ]             │ true     │
│ 2       │ new ObjectId('6aed77be3330be6b59f2b7c0') │ 'Pedro Reyes'    │ 19  │ 'BSCS'   │ [ 'Programming', 'Math' ]    │ true     │
│ 3       │ new ObjectId('6aed77be3330be6b59f2b7c1') │ 'Ana Lim'        │ 22  │ 'BSIT'   │ [ 'Web Dev' ]                │ false    │
└─────────┴──────────────────────────────────────────┴──────────────┴─────┴──────────┴──────────────────────────────┴──────────┘
BSCS students only:
┌─────────┬──────────────────────────────────────────┬──────────────┬─────┬──────────┬──────────────────────────────┬──────────┐
│ (index) │ _id                                      │ name         │ age │ course   │ subjects                     │ enrolled │
├─────────┼──────────────────────────────────────────┼──────────────┼─────┼──────────┼──────────────────────────────┼──────────┤
│ 0       │ new ObjectId('6aed77be3330be6b59f2b7be') │ 'Juan Dela Cruz' │ 20  │ 'BSCS'   │ [ 'Programming', 'Databases' ] │ true     │
│ 1       │ new ObjectId('6aed77be3330be6b59f2b7c0') │ 'Pedro Reyes'    │ 19  │ 'BSCS'   │ [ 'Programming', 'Math' ]    │ true     │
└─────────┴──────────────────────────────────────────┴──────────────┴─────┴──────────┴──────────────────────────────┴──────────┘
Students older than 20:
┌─────────┬──────────────────────────────────────────┬──────────────┬─────┬──────────┬──────────────────┬──────────┐
│ (index) │ _id                                      │ name         │ age │ course   │ subjects         │ enrolled │
├─────────┼──────────────────────────────────────────┼──────────────┼─────┼──────────┼──────────────────┼──────────┤
│ 0       │ new ObjectId('6aed77be3330be6b59f2b7bf') │ 'Maria Santos' │ 21  │ 'BSIT'   │ [ 'Networking' ] │ true     │
│ 1       │ new ObjectId('6aed77be3330be6b59f2b7c1') │ 'Ana Lim'      │ 22  │ 'BSIT'   │ [ 'Web Dev' ]    │ false    │
└─────────┴──────────────────────────────────────────┴──────────────┴─────┴──────────┴──────────────────┴──────────┘
findOne result: {
  _id: new ObjectId('6aed77be3330be6b59f2b7bf'),
  name: 'Maria Santos',
  age: 21,
  course: 'BSIT',
  subjects: [ 'Networking' ],
  enrolled: true
}
Names and courses only (projection):
┌─────────┬──────────────┬──────────┐
│ (index) │ name         │ course   │
├─────────┼──────────────┼──────────┤
│ 0       │ 'Juan Dela Cruz' │ 'BSCS' │
│ 1       │ 'Maria Santos'   │ 'BSIT' │
```

Figure 2 — CREATE (insertOne/insertMany added 4 students) and READ (find all, filter by course, age > 20, findOne, and projection of selected fields).



```
node crud-demo.js — UPDATE & DELETE

│ 2       │ 'Pedro Reyes'    │ 'BSCS' │
│ 3       │ 'Ana Lim'        │ 'BSIT' │
└─────────┴──────────────────┴────────┘

===== UPDATE: updateOne & updateMany =====
updateOne -> matched 1, modified 1
updateMany -> matched 3, modified 3
After updates:
┌─────────┬──────────────────────────────────────────┬──────────────┬─────┬──────────┬──────────────────────────────────┬──────────┬────────────┐
│ (index) │ _id                                      │ name         │ age │ course   │ subjects                         │ enrolled │ yearLevel  │
├─────────┼──────────────────────────────────────────┼──────────────┼─────┼──────────┼──────────────────────────────────┼──────────┼────────────┤
│ 0       │ new ObjectId('6aed77be3330be6b59f2b7be') │ 'Juan Dela Cruz' │ 20  │ 'BSCS'   │ [ 'Programming', 'Databases' ]   │ true     │ 2          │
│ 1       │ new ObjectId('6aed77be3330be6b59f2b7bf') │ 'Maria Santos'   │ 22  │ 'BSCS'   │ [ 'Networking' ]                 │ true     │ 2          │
│ 2       │ new ObjectId('6aed77be3330be6b59f2b7c0') │ 'Pedro Reyes'    │ 19  │ 'BSCS'   │ [ 'Programming', 'Math', 'NoSQL Databases' ] │ true     │ 2          │
│ 3       │ new ObjectId('6aed77be3330be6b59f2b7c1') │ 'Ana Lim'        │ 22  │ 'BSIT'   │ [ 'Web Dev' ]                    │ false    │            │
└─────────┴──────────────────────────────────────────┴──────────────┴─────┴──────────┴──────────────────────────────────┴──────────┴────────────┘
```

Figure 3 — UPDATE (updateOne changed Maria's course/age, updateMany added yearLevel to all enrolled students, $push added a subject) and DELETE (deleteOne removed one student, deleteMany removed all non-enrolled students), leaving 2 documents.

5. Running the System — Web Application

Running `npm run dev` starts the app at http://localhost:3000. The page calls the REST API, which performs the same CRUD operations on MongoDB:

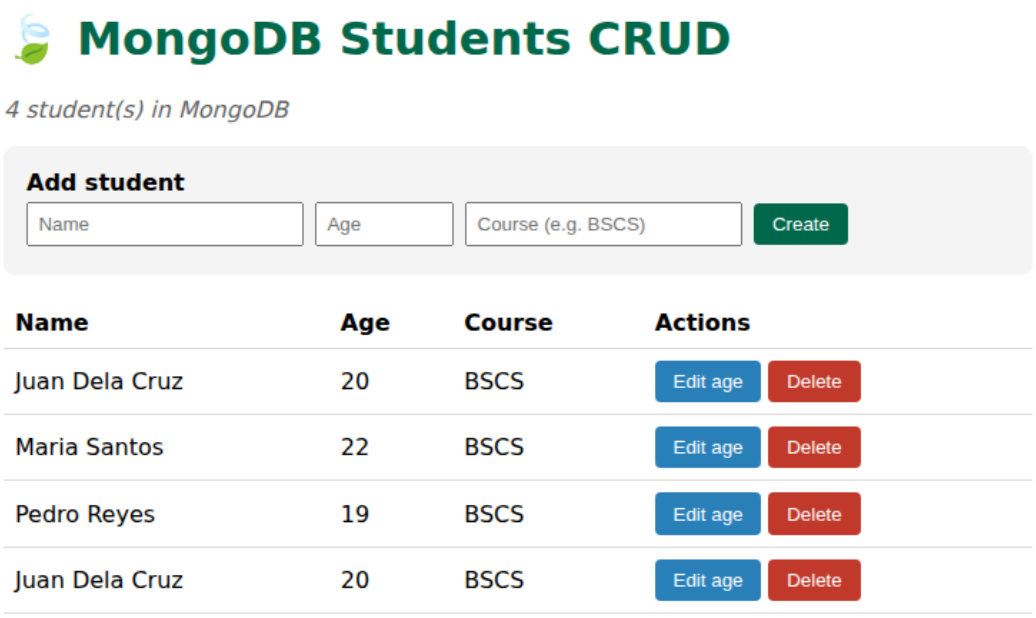


Figure 4 — The web app on load: it reads all students from MongoDB through GET /api/students and displays them in a table.

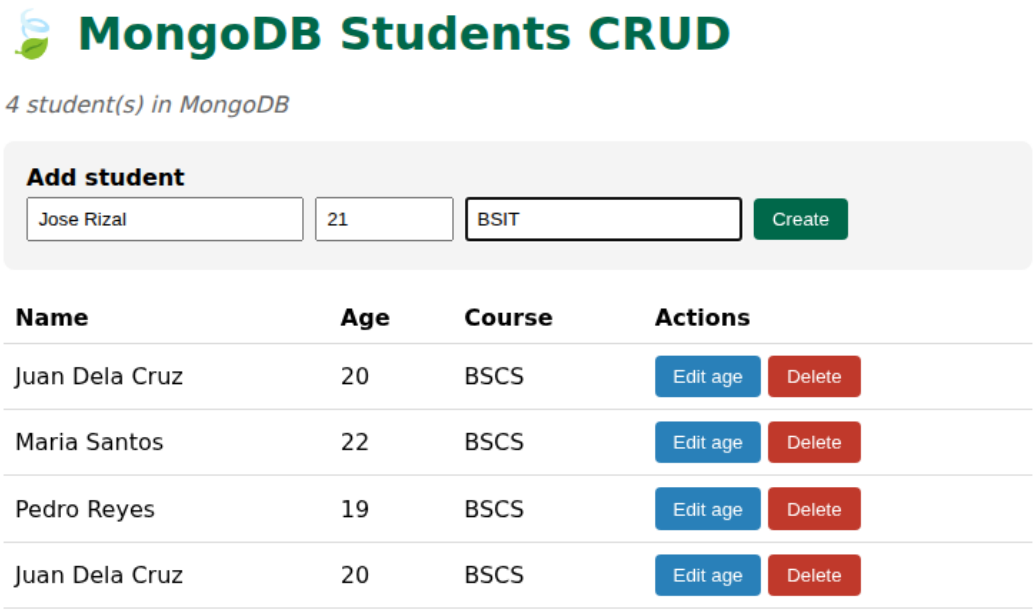


Figure 5 — CREATE: filling in the Add Student form (name, age, course).

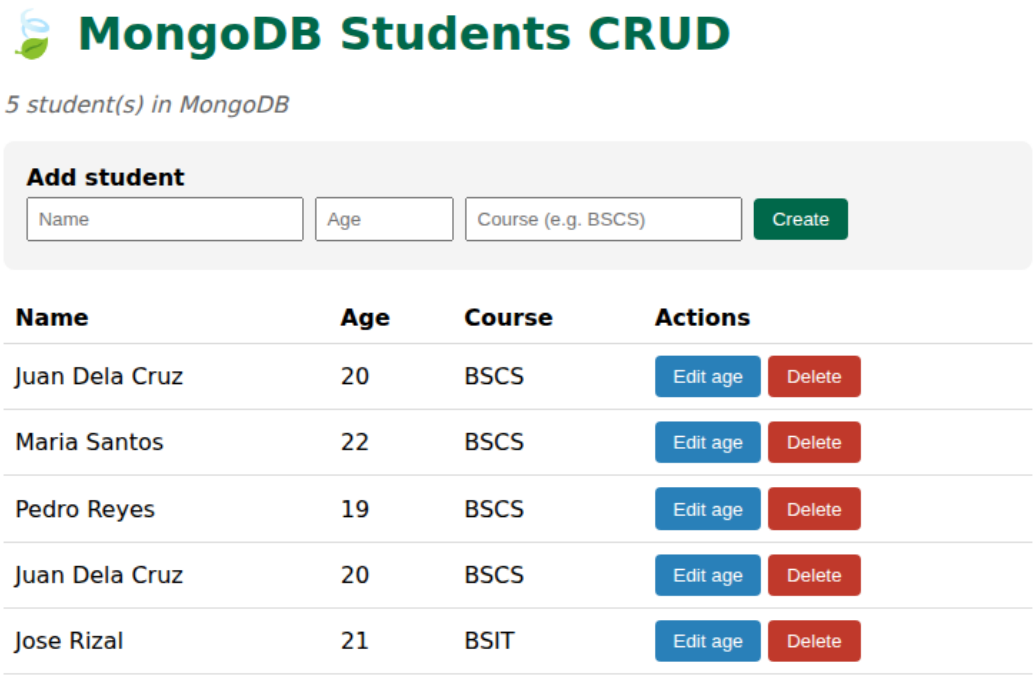


Figure 6 — After clicking Create: the new student (Jose Rizal) was inserted into MongoDB via POST /api/students and now appears in the table (5 students).

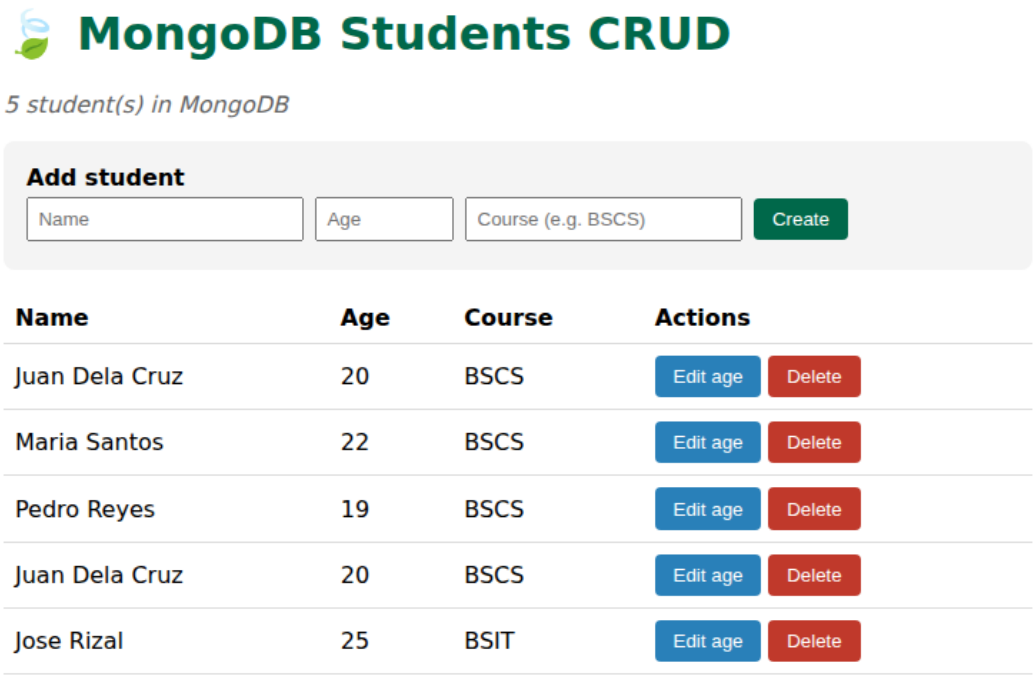


Figure 7 — UPDATE: the Edit age button sends PUT /api/students with the new value; MongoDB updates the document and the table reloads.

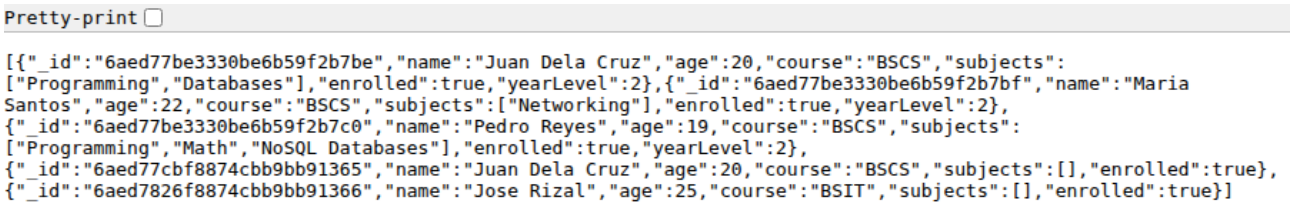


Figure 8 — The raw API response (GET /api/students): the same documents exactly as stored in the MongoDB collection, returned as JSON. (DELETE works the same way — the Delete button calls DELETE /api/students?id=... and the row disappears.)

6. How to Run

1. Start MongoDB: `docker start mongodb`
2. Console demo: `npm run demo`
3. Web app: `npm run dev` then open http://localhost:3000
4. Cloud deployment: push to GitHub and import into Vercel with MONGODB_URI (MongoDB Atlas) as an environment variable.